LAYER 5 · SESSION

L5

Sessions, Tokens and Access

Remembering who someone is between requests, and deciding what they may do: login sessions, JWTs, Redis, roles and keeping tenants apart.

YASIR ABDULLAH · YASIR-PORTFOLIO-ZETA.VERCEL.APP/NOTES/EBOOKS/LAYER-5-SESSION

SKIP TO CONTENT

Contents

1 **Login Sessions**

How a server remembers a signed-in user with a random ID in a cookie.

2 **JSON Web Tokens**

A signed token that carries its own claims, and the rules for using one safely.

3 **Redis as a Session Store**

Keeping sessions in one fast shared store so any app server can answer any request.

4 **Role-Based Access Control**

Giving permissions to roles, not people, and checking them on the server every time.

5 **Tenant Isolation**

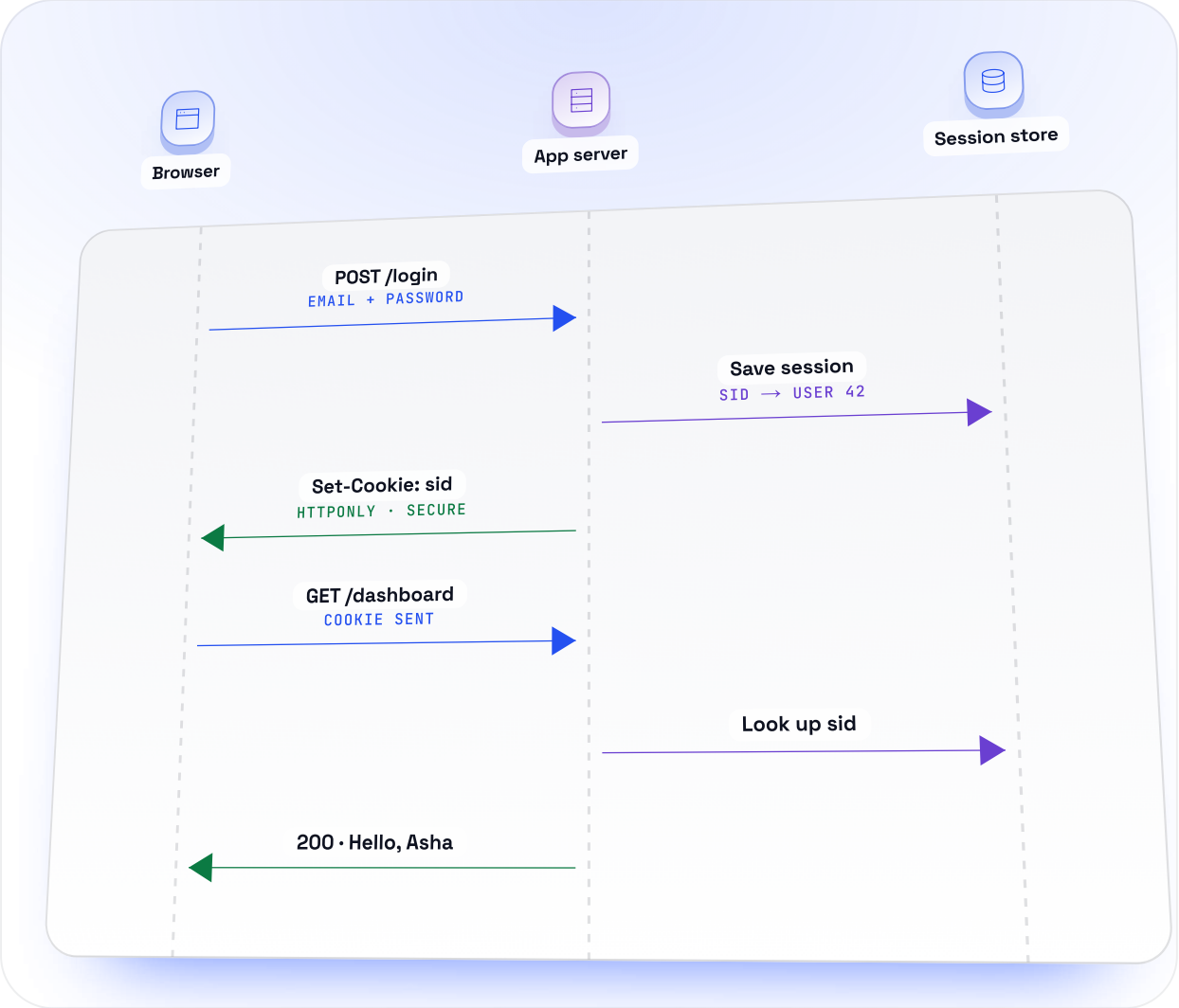
Making sure one customer's data can never be read by another, in an app they all share.

SKIP TO CONTENT

CHAPTER 1 · LOGIN SESSIONS

Login Sessions

How a server remembers a signed-in user with a random ID in a cookie.



The password is checked once. After that the browser shows a random session ID, and the server looks it up on every request.

HTTP forgets everything between requests. A login session is how a server remembers that the person asking for `/dashboard` is the same person who typed the right password a minute ago.

How a session works

SKIP TO CONTENT

s their email and password once.

ks the password against a slow, salted hash (Argon2 or bcrypt), never a plain copy.

3. It creates a **session**: a long random ID mapped to the user, stored on the server.
4. It sends the ID back in a cookie.
5. On every later request the browser sends the cookie, and the server looks the ID up.

The session ID itself means nothing. It is just a key; all the real information stays on the server.

Cookie settings that matter

```
TEXT

Set-Cookie: sid=Hk3…9pQ; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age
```

ATTRIBUTE	WHY
`HttpOnly`	JavaScript cannot read it, so an XSS bug cannot steal it
`Secure`	Only sent over HTTPS
`SameSite`	Not sent on most cross-site requests, which blocks CSRF
`Max-Age`	The session ends on its own

Rules for safe sessions

- Generate IDs from a cryptographically secure random source, at least 128 bits.
- **Issue a new session ID at login**, so an ID planted before login is useless (session fixation).
- Expire sessions after inactivity, and absolutely after a fixed time.
- On logout, delete the session **on the server**, not just the cookie.
- Store only a hash of the session ID in the database, so a leaked table cannot be replayed.
- ttempts per account and per address.

SKIP TO CONTENT

Sessions or tokens?

Server-side sessions are simple and can be revoked instantly: delete the row and the user is out. Self-contained tokens such as JWTs avoid a lookup but are hard to revoke before they expire. For a normal web app with its own backend, sessions are usually the better choice.

When it goes wrong

- **Users logged out at random:** sessions are stored in one server's memory and the load balancer sends them to another server. Use a shared store such as Redis or the database.
- **Logged in on HTTP, logged out on HTTPS:** a `Secure` cookie is not sent over HTTP. Redirect everything to HTTPS.
- **Login works locally, not in production:** the cookie's domain or `SameSite` setting does not match the real site.

SKIP TO CONTENT

CHAPTER 2 · JWT

JSON Web Tokens

A signed token that carries its own claims, and the rules for using one safely.



Three base64url parts joined by dots. Anyone can read the payload; only the signature proves it was not changed.

A JSON Web Token is a small, signed piece of JSON. Instead of storing a session on the server, the server hands the client a token that carries the facts itself, such as who the user is and when the token expires, plus a signature that proves nobody changed them.

The three parts

A JWT is three base64url strings joined by dots:

```
TEXT
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0MiIsInJvbGUiOiJzdGFmZiIsImV4cCI6MTczMDA
```

1. **Header:** the signing algorithm, for example `{"alg": "HS256"}`.
2. **Payload:** the claims.
3. **Signature:** computed over the header and payload with a secret key (HS256) or a private key (RS256, ES256).

SKIP TO CONTENT

```
JSON
```

```
{
  "sub": "42",
  "role": "staff",
  "iat": 1730000000,
  "exp": 1730000900
}
```

`sub` is the subject (the user), `iat` when it was issued, `exp` when it expires.

> **WARNING**
>
> The payload is **encoded, not encrypted**. Anyone holding the token can read it. Never put passwords, personal details or secrets in a JWT.

Verifying a token

On every request, the server must:

1. Check the signature with the expected key and **the expected algorithm only**. Reject `alg: none`.
2. Check `exp` (and `nbf` if present).
3. Check `iss` and `aud` if you use them.
4. Only then trust the claims.

TS

```ts
import { jwtVerify } from "jose";

const { payload } = await jwtVerify(token, secret, {
  algorithms: ["HS256"],
  issuer: "https://api.example.com",
});
```

The revocation problem

alid until it expires, even if the user logs out or is disabled.

SKIP TO CONTENT

- Keep **access tokens short-lived**, around 5 to 15 minutes.
- Use a longer-lived **refresh token**, stored server-side so it can be revoked, to get new access tokens.
- Keep a deny-list for the rare emergency.

Where to keep it in a browser

An `HttpOnly`, `Secure` cookie is safer than `localStorage`, which any script on the page, including an injected one, can read.

When it goes wrong

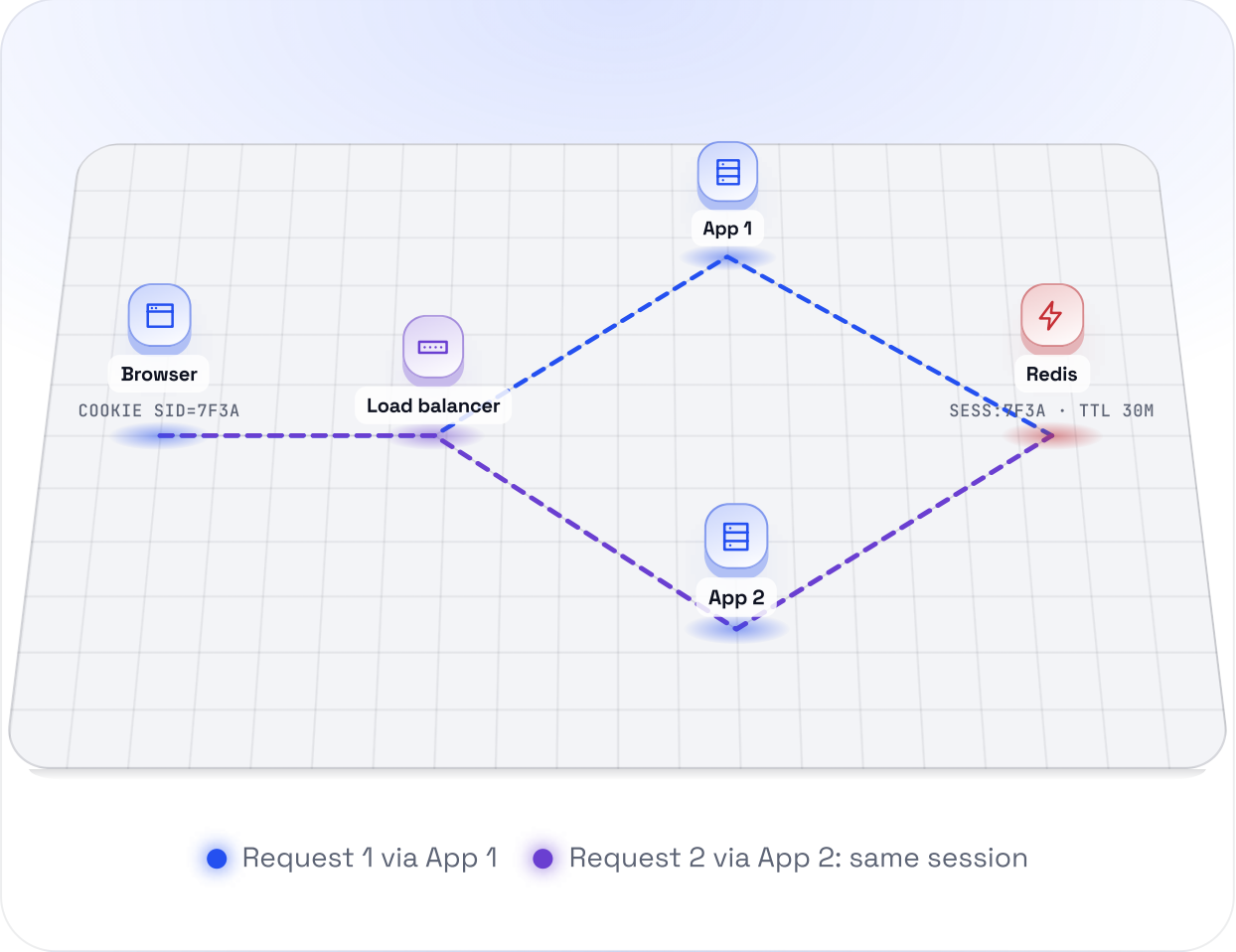
- **"Invalid signature" after a deploy:** the signing secret changed or differs between servers.
- **Tokens rejected a few minutes early or late:** the server clocks disagree. Sync them with NTP and allow a little leeway.
- **User still has access after being removed:** the access token has not expired yet.

SKIP TO CONTENT

CHAPTER 3 · REDIS SESSION STORE

Redis as a Session Store

Keeping sessions in one fast shared store so any app server can answer any request.



Two requests from one user land on different app servers. Both find the same session in Redis, so the user stays signed in.

If sessions live in an app server's memory, they vanish on every restart and only that one server knows about them. Put two servers behind a load balancer and users are logged out whenever a request lands on the other one. A shared session store fixes both problems, and **Redis** is the usual choice.

Why Redis fits

- kup takes well under a millisecond.
- very key can have a time to live, so old sessions delete themselves.

SKIP TO CONTENT

- **Shared:** every app server talks to the same Redis, so any server can answer any request.
- **Simple:** a session is just a key and a value.

The shape of a session

```shell
SET sess:7f3a9c… '{"userId":42,"role":"staff","tenant":"clinic-a"}' EX 1
GET sess:7f3a9c…
TTL sess:7f3a9c…
DEL sess:7f3a9c…
```

- `EX 1800` expires the key after 30 minutes.
- Refresh the TTL on each request for a sliding timeout.
- `DEL` on logout ends the session instantly, everywhere.

In a Node.js app

```ts
import { createClient } from "redis";

const redis = createClient({ url: process.env.REDIS_URL });
await redis.connect();

export async function saveSession(id: string, data: object) {
  await redis.set(`sess:${id}`, JSON.stringify(data), { EX: 1800 });
}

export async function readSession(id: string) {
  const raw = await redis.get(`sess:${id}`);
  return raw ? JSON.parse(raw) : null;
}
```

SKIP TO CONTENT

fe and healthy

- Never expose Redis to the internet. Bind it to a private network, require a password (or ACL users) and use TLS between servers.
- Set `maxmemory` and an eviction policy, and watch memory use. A full Redis that cannot evict will refuse writes.
- Decide what happens if Redis restarts: with persistence (AOF or RDB snapshots) sessions survive; without it, everyone logs in again, which may be acceptable.
- Prefix keys (`sess:`, `rate:`, `cache:`) so different uses never collide.

Checking it

```shell
redis-cli ping
redis-cli info memory
redis-cli --scan --pattern 'sess:*' | head
```

When it goes wrong

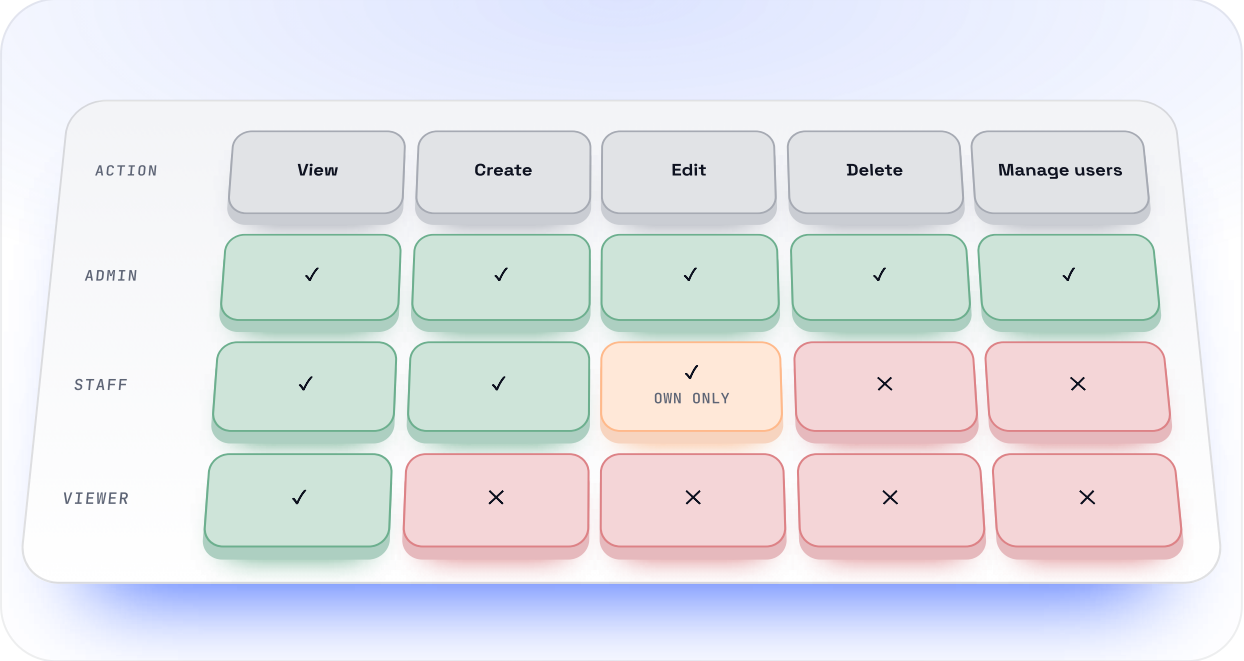
- **Everyone logged out at once:** Redis restarted without persistence, or memory filled and sessions were evicted.
- **Slow logins:** the app opens a new Redis connection per request. Reuse one client.
- **Sessions never expire:** they were written without a TTL. Check with `TTL key`; `-1` means none.

SKIP TO CONTENT

CHAPTER 4 · ROLE-BASED ACCESS

Role-Based Access Control

Giving permissions to roles, not people, and checking them on the server every time.



Each role has a fixed set of actions. A user gets a role; the server checks the role on every request.

Authentication answers "who are you?" Authorisation answers "what may you do?" Role-based access control (RBAC) answers the second question by giving permissions to **roles** and giving each user a role, instead of setting permissions person by person.

Users, roles, permissions

- A **permission** is one action on one kind of thing: `ticket:view`, `ticket:delete`, `user:manage`.
- A **role** is a named set of permissions: Admin, Staff, Viewer.
- A **user** has one or more roles.

When someone changes job, you change their role. When a new feature arrives, ion to the right roles once.

SKIP TO CONTENT

PERMISSION	ADMIN	STAFF	VIEWER
View tickets	✓	✓	✓
Create ticket	✓	✓	
Edit ticket	✓	own only	
Delete ticket	✓		
Manage users	✓		

”Own only” is where RBAC meets ownership: the role allows editing, and a second check confirms the record belongs to the user.

Check on the server, every time

Hiding a button is not security. The server must check the permission on every request, because anyone can send the request without the button.

TS

```ts
const PERMISSIONS = {
  ADMIN: ["ticket:view", "ticket:create", "ticket:edit", "ticket:delete'
  STAFF: ["ticket:view", "ticket:create", "ticket:edit"],
  VIEWER: ["ticket:view"],
} as const;

export function can(role: keyof typeof PERMISSIONS, permission: string)
  return (PERMISSIONS[role] as readonly string[]).includes(permission);
}

// In the handler
if (!can(user.role, "ticket:delete")) {
  return new Response("Forbidden", { status: 403 });
}
```

SKIP TO CONTENT

- **Least privilege:** give the smallest role that lets someone do their job.

- **Deny by default:** a permission that is not granted is refused.
- **Few roles:** a handful of clear roles beats dozens of near-copies.
- **Separate duties:** the person who approves a payment should not be the one who creates it.
- **Log** permission changes and denied attempts.

401 or 403?

- **401 Unauthorized:** not signed in, or the session is invalid. Sign in and try again.
- **403 Forbidden:** signed in, but this role may not do this.

When it goes wrong

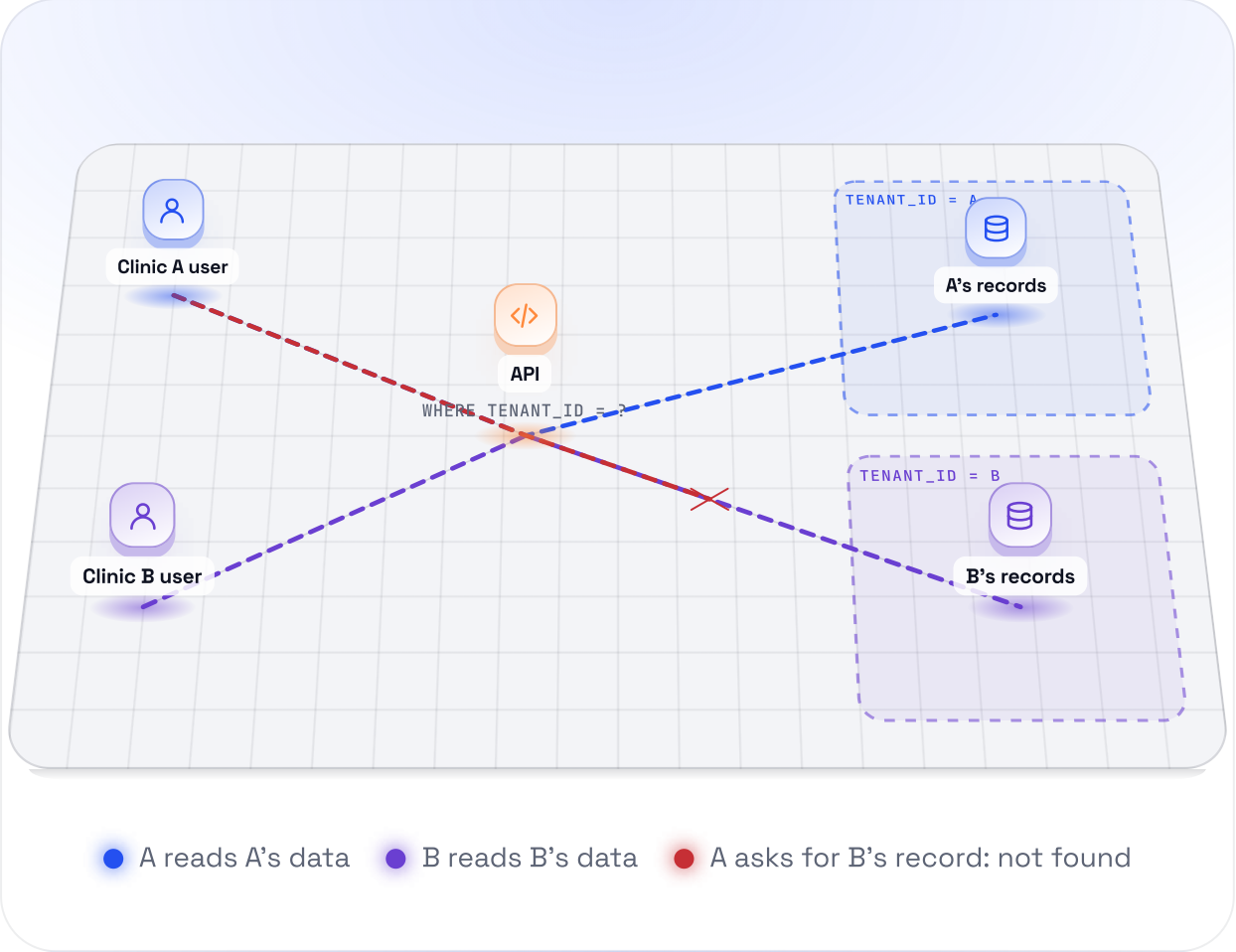
- **A user can see a page but every action fails:** the page checks one permission and the API another. Use one shared `can()` function.
- **Everyone ends up Admin:** roles were too narrow, so people were promoted to get work done. Add the missing permission to the right role instead.

SKIP TO CONTENT

CHAPTER 5 · TENANT ISOLATION

Tenant Isolation

Making sure one customer's data can never be read by another, in an app they all share.



Every query is scoped to the tenant in the signed-in session. A request for another tenant's record finds nothing.

A multi-tenant app serves many customers, the tenants, from one codebase and usually one database. Clinic A and Clinic B both use the same system. **Tenant isolation** is the guarantee that A can never read, change or even detect B's data, whatever URL or ID they send.

Ways to separate tenants

SKIP TO CONTENT

MODEL	HOW	TRADE-OFF
Shared tables	A `tenant_id` column on every row	Cheapest; isola
Schema per tenant	One schema each in the same database	Stronger; hard
Database per tenant	Separate databases	Strongest; mo

Most apps start with shared tables. That makes the next rule non-negotiable.

Every query is scoped

The tenant comes from the **signed-in session**, never from the request body, query string or a header the client controls.

```ts
// Wrong: trusts the ID alone
await db.patient.findUnique({ where: { id } });

// Right: the record must also belong to the caller's tenant
await db.patient.findFirst({ where: { id, tenantId: session.tenantId } }
```

If the record belongs to another tenant, the query finds nothing and the API returns **404**, not 403. A 403 would confirm that the record exists.

Make the safe path the easy path

Relying on every developer to remember `tenantId` on every query will fail eventually. Add a layer that does it for them:

- A data-access helper or repository that always adds the tenant filter.
- An ORM extension or middleware that injects `tenantId` into every query.
- **PostgreSQL row-level security**, so the database itself refuses rows from other tenants:

```text
ent ENABLE ROW LEVEL SECURITY;
tenant_only ON patient
```

SKIP TO CONTENT

```
  USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

Beyond the database

- **Files:** prefix storage paths with the tenant ID and check it before serving.
- **Caches:** include the tenant in every cache key, or one clinic may see another's cached page.
- **Background jobs:** carry the tenant ID with the job and scope everything the job touches.
- **Logs and exports:** never mix tenants in one export.

Test it on purpose

Write tests that sign in as tenant A and request B's records by ID, through every endpoint. Each one must return nothing. Run them on every change.

When it goes wrong

- **A tenant sees another's name in a dropdown:** a lookup query forgot the filter.
- **A report shows totals across all clinics:** an aggregate query ran without the tenant scope.